Numerical Masterclass

— Diagram से —

(Day 1-10 के सारे numericals)

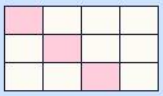
BPSC TRE 4.0 · Computer Science (Class 11-12) · Nitin के लिए

इस booklet में (हर chapter: Concept → पूरा हल किया Example → 2 Practice → अगले page पर हल)



1. CPU Scheduling — page 2

FCFS, SJF, SRTF, Priority, Round Robin — Gantt chart + CT/TAT/WT table



2. Page Replacement — page 9

FIFO, LRU, Optimal — frame grid, Hit/Fault, Belady's Anomaly



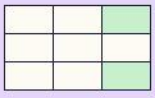
3. Disk Scheduling — page 13

FCFS, SSTF, SCAN, C-SCAN, LOOK (+C-LOOK) — track diagram + seek total



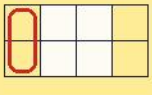
4. Number System + Complements — page 18

Ladder से conversion, fractions, grouping, 1's/2's complement, subtraction



5. SOP और POS — page 22

Truth table → minterm/maxterm, Σm ↔ ΠM, expression expand



6. K-map — page 25

2/3/4 variable, wrap-around, corners, octet, don't care, POS grouping

कैसे पढ़ें:

1) पहले Example का हर step pen-paper पर खुद बनाओ 2) Practice बिना देखे हल करो 3) फिर Solution page से match करो

⚠ गुलाबी Trap box = exam में यहीं गलती होती है — इन्हें दो बार पढ़ो

1. CPU Scheduling: कौन पहले चलेगा?

CPU एक समय में एक ही process चलाता है। कई process लाइन में खड़े हों तो OS (Operating System) तय करता है कि पहले किसको CPU मिले — इसी को CPU Scheduling कहते हैं। इसे हम Gantt Chart (समय की पट्टी) बनाकर हल करते हैं।

याद करने वाले शब्द + Formula

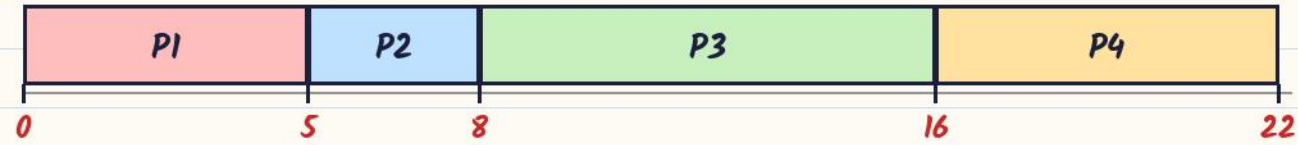
AT (Arrival Time) = process कब आया · BT (Burst Time) = उसे कितना CPU समय चाहिए
CT (Completion Time) = कब खत्म हुआ — Gantt chart से पढ़ो
TAT (Turnaround Time) = CT − AT → आने से जाने तक कुल समय
WT (Waiting Time) = TAT − BT → कुल समय में से काम का समय घटाओ = लाइन में इंतज़ार

हमारा Example (पूरे chapter में यही 4 process):

Process	AT (Arrival Time)	BT (Burst Time)	Priority
P1	0	5	2
P2	1	3	4
P3	2	8	1
P4	3	6	3

FCFS (First Come First Serve) = जो पहले आया, वो पहले चला

1. AT के क्रम में लगाओ: P1(0) → P2(1) → P3(2) → P4(3)
2. Gantt chart: हर box = एक process, नीचे के अंक = समय



3. हर box का end time = उस process का CT (Completion Time)

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	5	5	5 − 0 = 5	5 − 5 = 0
P2	1	3	8	8 − 1 = 7	7 − 3 = 4
P3	2	8	16	16 − 2 = 14	14 − 8 = 6
P4	3	6	22	22 − 3 = 19	19 − 6 = 13

AT = Arrival Time, BT = Burst Time, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

4. Avg TAT = (5 + 7 + 14 + 19) / 4 = 45/4 = 11.25
   Avg WT = (0 + 4 + 6 + 13) / 4 = 23/4 = 5.75

⚠ Trap: TAT = CT − AT है, CT − BT नहीं! और WT = TAT − BT। FCFS में एक लंबा process आगे हो तो पीछे वाले सब अटक जाते हैं — इसे Convoy Effect कहते हैं (exam में नाम पूछते हैं)।

1b. SJF (Shortest Job First) — Non-preemptive

Rule: जो process ready हैं, उनमें सबसे छोटा BT (Burst Time) वाला पहले। Non-preemptive = एक बार CPU मिल गया तो पूरा खत्म करके ही छोड़ेगा। इसलिए फ़ैसला सिर्फ़ तब होता है जब CPU खाली होता है।

Same example: P1(AT 0, BT 5), P2(AT 1, BT 3), P3(AT 2, BT 8), P4(AT 3, BT 6)

1 हर बार CPU खाली होने पर: ready कौन हैं? उनमें छोटा BT कौन?

t = 0	Ready:	P1(BT 5)	→ P1 (सबसे छोटा BT)
t = 5	Ready:	P2(BT 3), P3(BT 8), P4(BT 6)	→ P2 (सबसे छोटा BT)
t = 8	Ready:	P3(BT 8), P4(BT 6)	→ P4 (सबसे छोटा BT)
t = 14	Ready:	P3(BT 8)	→ P3 (सबसे छोटा BT)

क्यों? t = 0 पर सिर्फ़ P1 आया था, इसलिए P1 चला — भले ही बाद में छोटे आए, वो बीच में नहीं रुकेगा।

2 Gantt chart बनाओ



3 Table भरो (CT Gantt से, फिर formula)

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	5	5	5 − 0 = 5	5 − 5 = 0
P2	1	3	8	8 − 1 = 7	7 − 3 = 4
P3	2	8	22	22 − 2 = 20	20 − 8 = 12
P4	3	6	14	14 − 3 = 11	11 − 6 = 5

AT = Arrival Time, BT = Burst Time, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

Avg TAT = (5 + 7 + 20 + 11) / 4 = 43/4 = 10.75
Avg WT = (0 + 4 + 12 + 5) / 4 = 21/4 = 5.25

⚠ Trap: अगर दो process का BT बराबर हो तो जो पहले आया (कम AT) उसे लो। SJF का Avg WT सबसे कम होता है (optimal), पर लंबे process को Starvation (भूखा रहना) हो सकता है।

Ic. SRTF (Shortest Remaining Time First) — Preemptive

SRTF = SJF का preemptive रूप। Preemptive = नया process आते ही दोबारा check करो: जिसका बचा हुआ समय (Remaining Time) सबसे कम है, CPU उसी को — चल रहे process को बीच में रोक भी सकते हैं।

Same example: P1(AT 0, BT 5), P2(AT 1, BT 3), P3(AT 2, BT 8), P4(AT 3, BT 6)

1 हर arrival / completion पर रुककर Remaining Time देखो (bracket में):

t = 0	Ready:	P1(5) → P1 (कम बचा)
t = 1	Ready:	P1(4) P2(3) → P2 (कम बचा)
t = 2	Ready:	P1(4) P2(2) P3(8) → P2 (कम बचा)
t = 3	Ready:	P1(4) P2(1) P3(8) P4(6) → P2 (कम बचा)
t = 4	Ready:	P1(4) P3(8) P4(6) → P1 (कम बचा)
t = 8	Ready:	P3(8) P4(6) → P4 (कम बचा)
t = 14	Ready:	P3(8) → P3 (कम बचा)

t = 1 पर P1 के 4 बचे, P2 को 3 चाहिए → 3 < 4, इसलिए P1 को रोककर P2 चला (Preemption)।

2 Gantt chart (P1 दो टुकड़ों में चला)



3 Table: CT = process का आखिरी टुकड़ा जहाँ खत्म हुआ

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	5	8	8 − 0 = 8	8 − 5 = 3
P2	1	3	4	4 − 1 = 3	3 − 3 = 0
P3	2	8	22	22 − 2 = 20	20 − 8 = 12
P4	3	6	14	14 − 3 = 11	11 − 6 = 5

AT = Arrival Time, BT = Burst Time, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

Avg TAT = (8 + 3 + 20 + 11) / 4 = 42/4 = 10.5
Avg WT = (3 + 0 + 12 + 5) / 4 = 20/4 = 5

⚠ Trap: Preemptive में WT निकालने के लिए भी WT = TAT − BT ही लगाओ, टुकड़े गिनने की ज़रूरत नहीं। CT हमेशा आखिरी टुकड़े का end time होता है (P1 का CT = 8, 1 नहीं)।

Id. Priority Scheduling — Non-preemptive

Rule: ready process में जिसकी Priority सबसे ऊँची, वो पहले। यहाँ मान लो: छोटा नंबर = ऊँची priority (1 सबसे ऊपर)। Non-preemptive = चल रहा process पूरा होगा।

Process	AT (Arrival Time)	BT (Burst Time)	Priority
P1	0	5	2
P2	1	3	4
P3	2	8	1
P4	3	6	3

1 CPU खाली होने पर ready process की Priority देखो (Pr = Priority):

- t = 0 **Ready:** P1(Pr 2) → P1 (ऊँची priority)
- t = 5 **Ready:** P2(Pr 4) P3(Pr 1) P4(Pr 3) → P3 (ऊँची priority)
- t = 13 **Ready:** P2(Pr 4) P4(Pr 3) → P4 (ऊँची priority)
- t = 19 **Ready:** P2(Pr 4) → P2 (ऊँची priority)

2 Gantt chart



3 Table + averages

Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	5	5	5 − 0 = 5	5 − 5 = 0
P2	1	3	22	22 − 1 = 21	21 − 3 = 18
P3	2	8	13	13 − 2 = 11	11 − 8 = 3
P4	3	6	19	19 − 3 = 16	16 − 6 = 10

AT = Arrival Time, BT = Burst Time, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

Avg TAT = (5 + 21 + 11 + 16) / 4 = 53/4 = 13.25
Avg WT = (0 + 18 + 3 + 10) / 4 = 31/4 = 7.75

⚠ Trap: Question ध्यान से पढ़ो — कहीं 'बड़ा नंबर = ऊँची priority' भी दिया होता है, तब उल्टा चुनो। Starvation का इलाज = Aging (इंतज़ार के साथ priority बढ़ाते जाओ)।

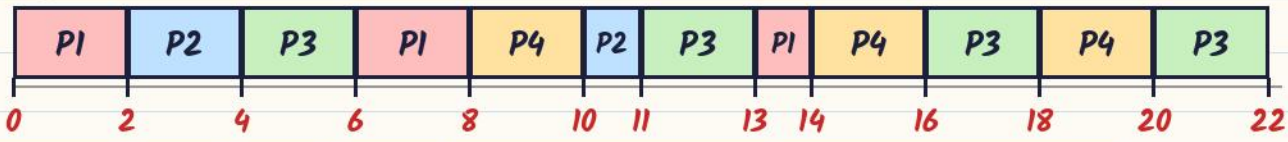
Ie. Round Robin (RR), Time Quantum = 2

RR = हर process को बारी-बारी से सिर्फ़ 2 unit (Time Quantum) मिलते हैं। काम बचा तो Ready Queue के पीछे चला जाता है। यह हमेशा preemptive है — Time Sharing system इसी पर चलते हैं।

1 हर quantum के बाद Ready Queue (आगे → पीछे):

t	Queue (पहले)	CPU	बचा BT	Queue (बाद में)
0	P1	P1: 0→2	3	P2 P3 P1
2	P2 P3 P1	P2: 2→4	1	P3 P1 P4 P2
4	P3 P1 P4 P2	P3: 4→6	6	P1 P4 P2 P3
6	P1 P4 P2 P3	P1: 6→8	1	P4 P2 P3 P1
8	P4 P2 P3 P1	P4: 8→10	4	P2 P3 P1 P4
10	P2 P3 P1 P4	P2: 10→11	0 ✓	P3 P1 P4
11	P3 P1 P4	P3: 11→13	4	P1 P4 P3
13	P1 P4 P3	P1: 13→14	0 ✓	P4 P3
14	P4 P3	P4: 14→16	2	P3 P4
16	P3 P4	P3: 16→18	2	P4 P3
18	P4 P3	P4: 18→20	0 ✓	P3
20	P3	P3: 20→22	0 ✓	खाली

t = 2 पर: P3 नया आया और P1 का quantum भी खत्म — नियम: नया आने वाला पहले queue में, फिर रुका हुआ P1।



Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	5	14	14 − 0 = 14	14 − 5 = 9
P2	1	3	11	11 − 1 = 10	10 − 3 = 7
P3	2	8	22	22 − 2 = 20	20 − 8 = 12
P4	3	6	20	20 − 3 = 17	17 − 6 = 11

AT = Arrival Time, BT = Burst Time, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

Avg TAT = (14 + 10 + 20 + 17) / 4 = 61/4 = 15.25
Avg WT = (9 + 7 + 12 + 11) / 4 = 39/4 = 9.75

⚠ Trap: Quantum बहुत बड़ा → RR = FCFS बन जाता है; बहुत छोटा → Context Switch ज़्यादा। बचा BT 2 से कम हो (जैसे 1) तो process 1 ही unit चलेगा, पूरा quantum नहीं।

If. Comparison + Practice Questions

एक ही example पर 5 algorithm — Avg WT (Waiting Time) की तुलना:



सीख:
SRTF/SJF का WT सबसे कम (छोटे काम पहले निपट गए)। RR का WT ज़्यादा, पर response अच्छा (सबको जल्दी बारी मिलती है)।

✍ Practice — खुद Gantt chart बनाओ, फिर अगला page देखो

Q1. SRTF (Shortest Remaining Time First) से Avg WT और Avg TAT निकालो:

Process	AT (Arrival Time)	BT (Burst Time)
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Q2. Round Robin, Time Quantum = 2 से Gantt chart, Avg TAT और Avg WT:

Process	AT (Arrival Time)	BT (Burst Time)
P1	0	4
P2	1	5
P3	2	2
P4	3	1

Hint:
Q1 में हर नए arrival (t = 2, 4, 5) पर रुककर Remaining Time compare करो। Q2 में हर quantum के बाद queue लिखो — नया आया process पहले, रुका हुआ बाद में।

1g. Practice Solutions (Diagram से)

Q1. SRTF (Shortest Remaining Time First) — हर रुकने पर Remaining Time:

t = 0	Ready:	P1(7)	→ P1
t = 2	Ready:	P1(5)	P2(4) → P2
t = 4	Ready:	P1(5)	P2(2)	P3(1) → P3
t = 5	Ready:	P1(5)	P2(2)	P4(4) → P2
t = 7	Ready:	P1(5)	P4(4) → P4
t = 11	Ready:	P1(5) → P1

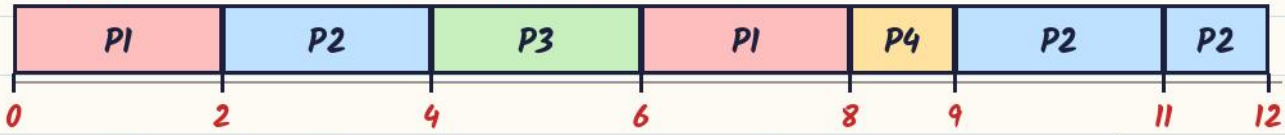


Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	7	16	16 − 0 = 16	16 − 7 = 9
P2	2	4	7	7 − 2 = 5	5 − 4 = 1
P3	4	1	5	5 − 4 = 1	1 − 1 = 0
P4	5	4	11	11 − 5 = 6	6 − 4 = 2

Avg TAT = (16 + 5 + 1 + 6) / 4 = 28/4 = 7
Avg WT = (9 + 1 + 0 + 2) / 4 = 12/4 = 3

Q2. RR (Round Robin), q = 2 — Queue हर step पर:

t=0	P1	P1 चला 0→2, बचा 2	बाद में: P2 P3 P1
t=2	P2 P3 P1	P2 चला 2→4, बचा 3	बाद में: P3 P1 P4 P2
t=4	P3 P1 P4 P2	P3 चला 4→6, बचा 0 ✓	बाद में: P1 P4 P2
t=6	P1 P4 P2	P1 चला 6→8, बचा 0 ✓	बाद में: P4 P2
t=8	P4 P2	P4 चला 8→9, बचा 0 ✓	बाद में: P2
t=9	P2	P2 चला 9→11, बचा 1	बाद में: P2
t=11	P2	P2 चला 11→12, बचा 0 ✓	बाद में: खाली



Process	AT	BT	CT	TAT = CT − AT	WT = TAT − BT
P1	0	4	8	8 − 0 = 8	8 − 4 = 4
P2	1	5	12	12 − 1 = 11	11 − 5 = 6
P3	2	2	6	6 − 2 = 4	4 − 2 = 2
P4	3	1	9	9 − 3 = 6	6 − 1 = 5

Avg TAT = (8 + 11 + 4 + 6) / 4 = 29/4 = 7.25
Avg WT = (4 + 6 + 2 + 5) / 4 = 17/4 = 4.25

AT = Arrival Time, BT = Burst Time, CT = Completion Time, TAT = Turnaround Time, WT = Waiting Time

2. Page Replacement: Frame भर गए, अब किसे निकालें?

RAM (Random Access Memory) में pages रखने के लिए गिने-चुने frame होते हैं। CPU जो page माँगे वो frame में मिल जाए = Hit; न मिले = Page Fault (disk से लाना पड़ेगा)। सारे frame भरे हों तो किसी एक को बाहर करना पड़ता है — कौन बाहर जाएगा, यही algorithm तय करता है।

3 नियम, 3 सवाल

FIFO (First In First Out): जो सबसे पहले आया था, वो बाहर — 'आने का क्रम' देखो
LRU (Least Recently Used): जिसे सबसे लंबे समय से use नहीं किया, वो बाहर — 'पीछे' देखो
Optimal (OPT): जो आगे सबसे देर से (या कभी नहीं) चाहिए, वो बाहर — 'आगे' देखो

Example (पूरे chapter में): Reference string, 3 frames

7 0 1 2 0 3 0 4 2 3 0 3 2

FIFO (First In First Out) — हर column = एक step

नया page आया (Fault) पहले से था (Hit)

Ref →	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2		0	0	0	0	3	3	3	2	2	2	2	2
Frame 3			1	1	1	1	0	0	0	3	3	3	3
Hit/Fault	F	F	F	F	H	F	F	F	F	F	F	H	H
बाहर गया				7		0	1	2	3	0	4		

Page Faults = 10 (लाल F गिनो) · Hits = 3 · Hit ratio = 3/13 · Miss ratio = 10/13

हर replacement क्यों? (क्रम = frame में आने का order)

- #4 ref 2: आने का क्रम: 7 → 0 → 1 ⇒ सबसे पुराना 7
- #6 ref 3: आने का क्रम: 0 → 1 → 2 ⇒ सबसे पुराना 0
- #7 ref 0: आने का क्रम: 1 → 2 → 3 ⇒ सबसे पुराना 1
- #8 ref 4: आने का क्रम: 2 → 3 → 0 ⇒ सबसे पुराना 2
- #9 ref 2: आने का क्रम: 3 → 0 → 4 ⇒ सबसे पुराना 3
- #10 ref 3: आने का क्रम: 0 → 4 → 2 ⇒ सबसे पुराना 0
- #11 ref 0: आने का क्रम: 4 → 2 → 3 ⇒ सबसे पुराना 4

⚠ Trap: पहले 3 reference (frames खाली) भी Fault गिने जाते हैं — इन्हें Compulsory/Cold miss कहते हैं। FIFO में Hit होने पर 'आने का क्रम' नहीं बदलता (LRU में बदलता है)।

2b. LRU (Least Recently Used) — पीछे देखो

Fault पर, frame में बैठे हर page का पिछला use कब हुआ — देखो। जिसका use सबसे पहले (सबसे पुराना) हुआ, वो बाहर। Hit होने पर उस page का 'last use' नया हो जाता है।

Same string, 3 frames: 7 0 1 2 0 3 0 4 2 3 0 3 2

नया page आया (Fault) पहले से था (Hit)

Ref →	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2		0	0	0	0	0	0	0	0	3	3	3	3
Frame 3			1	1	1	3	3	3	2	2	2	2	2
Hit/Fault	F	F	F	F	H	F	H	F	F	F	F	H	H
बाहर गया				7		1		2	3	0	4		

Page Faults = 9 (लाल F गिनो) · Hits = 4 · Hit ratio = 4/13 · Miss ratio = 9/13

हर replacement क्यों? (@ = reference string में position)

- **#4 ref 2:** पिछला use (position): 7@1, 0@2, 1@3 ⇒ सबसे पुराना use 7
- **#6 ref 3:** पिछला use (position): 2@4, 0@5, 1@3 ⇒ सबसे पुराना use 1
- **#8 ref 4:** पिछला use (position): 2@4, 0@7, 3@6 ⇒ सबसे पुराना use 2
- **#9 ref 2:** पिछला use (position): 4@8, 0@7, 3@6 ⇒ सबसे पुराना use 3
- **#10 ref 3:** पिछला use (position): 4@8, 0@7, 2@9 ⇒ सबसे पुराना use 0
- **#11 ref 0:** पिछला use (position): 4@8, 3@10, 2@9 ⇒ सबसे पुराना use 4

⚠ **Trap: Position 5 पर ref 0 Hit हुआ। Position 6 (ref 3) पर FIFO ने 0 निकाला (आने का क्रम), पर LRU ने 1 निकाला — क्योंकि Hit ने 0 को 'ताज़ा' कर दिया। इसलिए हर fault पर पीछे scan करो, सिर्फ़ load का क्रम नहीं।**

Shortcut: LRU Stack (ऊपर = अभी-अभी use, नीचे = सबसे पुराना)

हर reference को stack के ऊपर रखो (पहले से हो तो वहाँ से उठाकर ऊपर)। Fault पर जो सबसे नीचे है वही बाहर — scan की ज़रूरत नहीं।

Ref →	7	0	1	2	0	3	0	4	2	3	0	3	2
ऊपर (नया)	7	0	1	2	0	3	0	4	2	3	0	3	2
		7	0	1	2	0	3	0	4	2	3	0	3
नीचे (पुराना)			7	0	1	2	2	3	0	4	2	2	0

नारंगी (सबसे नीचे) वाला = अगले fault का victim। जैसे column 5 के बाद नीचे 1 है → ref 3 पर 1 बाहर ✓

2c. Optimal (OPT) — आगे देखो + Practice

Fault पर देखो कि frame के हर page की अगली ज़रूरत कब है। जो सबसे देर से (या कभी नहीं) आएगा, वो बाहर। यह सबसे कम fault देता है, पर असली OS भविष्य नहीं जानता — इसलिए सिर्फ़ तुलना (benchmark) के लिए।

नया page आया (Fault) पहले से था (Hit)

Ref →	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2
Frame 2		0	0	0	0	0	0	4	4	4	0	0	0
Frame 3			1	1	1	3	3	3	3	3	3	3	3
Hit/Fault	F	F	F	F	H	F	H	F	H	H	F	H	H
बाहर गया				7		1		0			4		

Page Faults = 7 (लाल F गिनो) · Hits = 6 · Hit ratio = 6/13 · Miss ratio = 7/13

#4 ref 2: अगला use (position): 7→कभी नहीं, 0→5, 1→कभी नहीं ⇒ सबसे दूर 7

#6 ref 3: अगला use (position): 2→9, 0→7, 1→कभी नहीं ⇒ सबसे दूर 1

#8 ref 4: अगला use (position): 2→9, 0→11, 3→10 ⇒ सबसे दूर 0

#11 ref 0: अगला use (position): 2→13, 4→कभी नहीं, 3→12 ⇒ सबसे दूर 4

तुलना (same string, 3 frames):
FIFO = 10 fault > LRU = 9 fault > Optimal = 7 fault (Optimal हमेशा सबसे कम)

⚠ Trap: Belady's Anomaly: FIFO में frame बढ़ाने पर fault कभी-कभी बढ़ जाते हैं! LRU और Optimal में ऐसा कभी नहीं होता (ये Stack algorithm हैं)।

✍ Practice (अगले page पर पूरा हल)

Q1. String 1 2 3 4 1 2 5 1 2 3 4 5 पर FIFO चलाओ — पहले 3 frames, फिर 4 frames। Faults गिनो। क्या Belady's Anomaly दिखी?

Q2. String 4 7 6 1 7 6 1 2 7 2 पर LRU, 3 frames — Page Faults और Hit ratio निकालो।

2d. Practice Solutions (Frame Grid)

Q1 (a) FIFO, 3 frames

Ref →	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1	1	1	4	4	4	5	5	5	5	5	5
Frame 2		2	2	2	1	1	1	1	1	3	3	3
Frame 3			3	3	3	2	2	2	2	2	4	4
Hit/Fault	F	F	F	F	F	F	F	H	H	F	F	H
बाहर गया				1	2	3	4			1	2	

Page Faults = 9 (लाल F गिनो) · Hits = 3 · Hit ratio = 3/12 · Miss ratio = 9/12

Q1 (b) FIFO, 4 frames

Ref →	1	2	3	4	1	2	5	1	2	3	4	5
Frame 1	1	1	1	1	1	1	5	5	5	5	4	4
Frame 2		2	2	2	2	2	2	1	1	1	1	5
Frame 3			3	3	3	3	3	3	2	2	2	2
Frame 4				4	4	4	4	4	4	3	3	3
Hit/Fault	F	F	F	F	H	H	F	F	F	F	F	F
बाहर गया							1	2	3	4	5	1

Page Faults = 10 (लाल F गिनो) · Hits = 2 · Hit ratio = 2/12 · Miss ratio = 10/12

हाँ, Belady's Anomaly ✓

3 frames → 9 fault, 4 frames → 10 fault। Frame बढ़ाए, फिर भी fault बढ़ गए (सिर्फ़ FIFO में होता है)।

Q2 LRU, 3 frames

Ref →	4	7	6	1	7	6	1	2	7	2
Frame 1	4	4	4	1	1	1	1	1	1	1
Frame 2		7	7	7	7	7	7	2	2	2
Frame 3			6	6	6	6	6	6	7	7
Hit/Fault	F	F	F	F	H	H	H	F	F	H
बाहर गया				4				7	6	

Page Faults = 6 (लाल F गिनो) · Hits = 4 · Hit ratio = 4/10 · Miss ratio = 6/10

#4 ref 1: पिछला use (position): 4@1, 7@2, 6@3 ⇒ सबसे पुराना use 4

#8 ref 2: पिछला use (position): 1@7, 7@5, 6@6 ⇒ सबसे पुराना use 7

#9 ref 7: पिछला use (position): 1@7, 2@8, 6@6 ⇒ सबसे पुराना use 6

3. Disk Scheduling: Head कहाँ-कहाँ घूमेगा?

Hard disk का head एक track से दूसरे track पर जाता है — इस दूरी को Seek Distance (कितने track चला) कहते हैं। कई request लाइन में हों तो किस क्रम में जाएँ कि head कम घूमे — यही Disk Scheduling है। हर कदम की दूरी = |नया track − पुराना track| (हमेशा positive), फिर सब जोड़ दो।

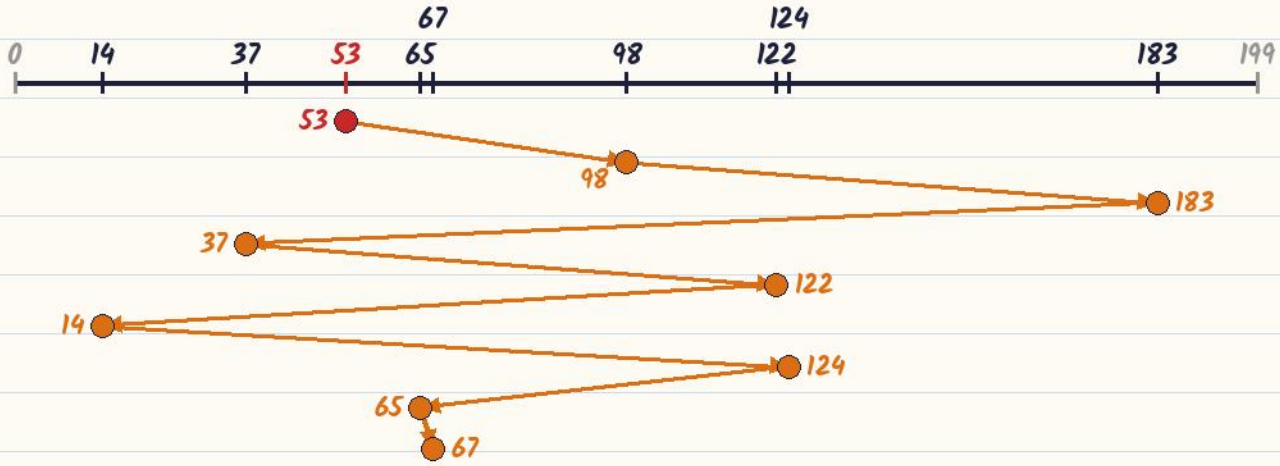
5 Algorithm, एक-एक line में

FCFS (First Come First Serve): जिस क्रम में request आई, उसी क्रम में
SSTF (Shortest Seek Time First): अभी जहाँ हो, उसके सबसे पास वाली request
SCAN (Elevator): एक दिशा में आखिरी track (0 या 199) तक, फिर पलटो
C-SCAN (Circular SCAN): एक दिशा में end तक, फिर सीधे दूसरे end पर कूदो, उसी दिशा में चलो
LOOK: SCAN जैसा, पर end तक नहीं — आखिरी request पर ही पलट जाओ

Example (पूरे chapter में):

Tracks 0–199 · Head = 53 पर · Queue = 98, 183, 37, 122, 14, 124, 65, 67 ·
(SCAN/C-SCAN/LOOK में head बड़े track की तरफ़ जा रहा है)

FCFS (First Come First Serve) — लाल बिंदु = शुरुआत (53)



53→98: |98−53| = 45
98→183: |183−98| = 85
183→37: |37−183| = 146
37→122: |122−37| = 85
122→14: |14−122| = 108
14→124: |124−14| = 110
124→65: |65−124| = 59
65→67: |67−65| = 2

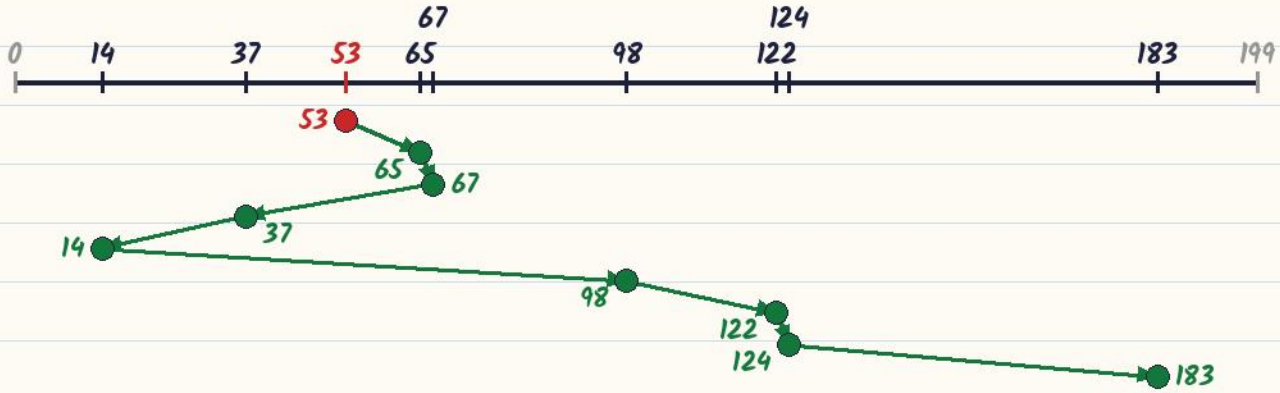
Total = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640

⚠ Trap: FCFS में कोई चालाकी नहीं — head 37 से 122 फिर 14 पर उछलता है, इसलिए दूरी सबसे ज़्यादा। Queue का क्रम बिल्कुल न बदलो।

3b. SSTF और SCAN

SSTF (Shortest Seek Time First)

हर बार: अभी जहाँ हो, वहाँ से सबसे पास की बची request। 53 से: 65 (12 दूर) < 37 (16 दूर) → 65।



53→65: |65−53| = 12 | 65→67: |67−65| = 2 | 67→37: |37−67| = 30

37→14: |14−37| = 23 | 14→98: |98−14| = 84 | 98→122: |122−98| = 24

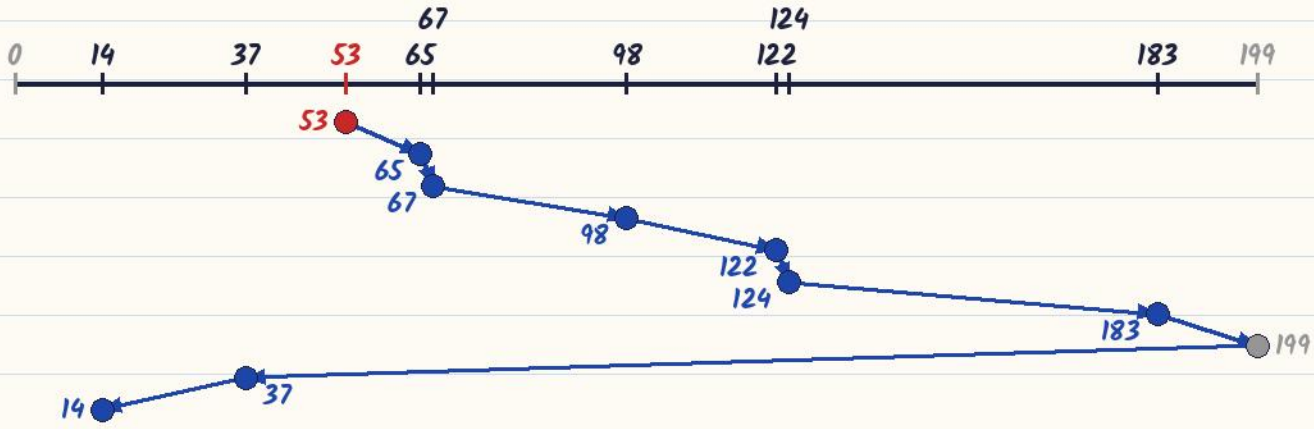
122→124: |124−122| = 2 | 124→183: |183−124| = 59

Total = 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236

⚠ Trap: 67 के बाद: 37 (30 दूर) बनाम 98 (31 दूर) → 37 पास है, तो पीछे मुड़ो! SSTF में Starvation हो सकता है (दूर वाली request रुकी रहती है)।

SCAN (Elevator) — ऊपर की तरफ़

53 से बढ़ते क्रम में सब request, फिर disk के आखिरी track 199 तक (भले वहाँ request न हो), फिर पलटकर 37, 14।



53→65: |65−53| = 12 | 65→67: |67−65| = 2 | 67→98: |98−67| = 31

98→122: |122−98| = 24 | 122→124: |124−122| = 2 | 124→183: |183−124| = 59

183→199: |199−183| = 16 | 199→37: |37−199| = 162 | 37→14: |14−37| = 23

Total = 12 + 2 + 31 + 24 + 2 + 59 + 16 + 162 + 23 = 331

⚠ Trap: SCAN में end (199) तक जाना ज़रूरी है — 183 पर नहीं पलटना (वो LOOK है)। 199 = tracks 0–199 का आखिरी, 200 नहीं!

3c. C-SCAN और LOOK

C-SCAN (Circular SCAN)

53 से ऊपर 199 तक, फिर 0 पर कूदो (नारंगी dotted), फिर ऊपर: 14, 37।



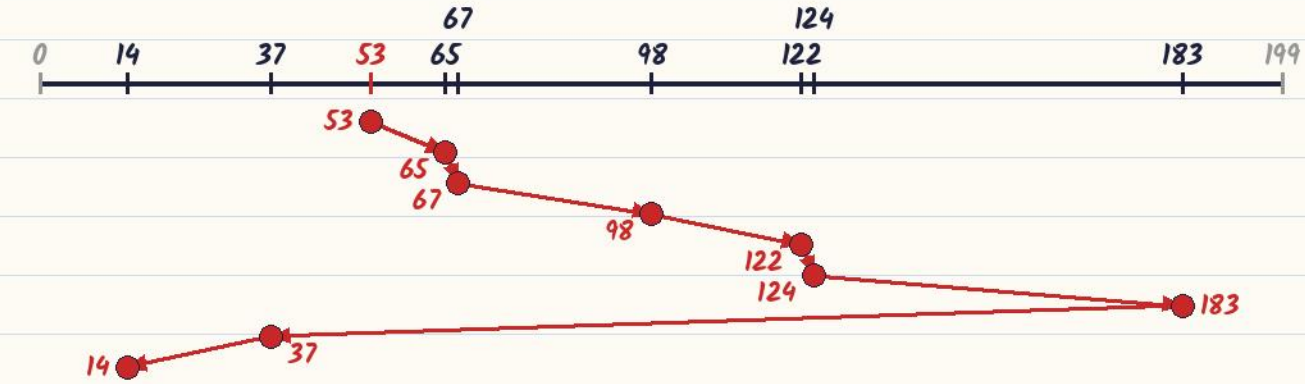
53→65: |65−53| = 12
65→67: |67−65| = 2
67→98: |98−67| = 31
98→122: |122−98| = 24
122→124: |124−122| = 2
124→183: |183−124| = 59
183→199: |199−183| = 16
199→0: |0−199| = 199
0→14: |14−0| = 14
14→37: |37−14| = 23

Total = 12 + 2 + 31 + 24 + 2 + 59 + 16 + 199 + 14 + 23 = 382

⚠ Trap: वापसी की छलांग (199 → 0 = 199) को गिनना है या नहीं — question देखो। यहाँ गिना है: 382। न गिनें तो 382 − 199 = 183।

LOOK

SCAN जैसा, पर आखिरी request 183 पर ही पलटो (199 तक नहीं), फिर 37, 14।



53→65: |65−53| = 12
65→67: |67−65| = 2
67→98: |98−67| = 31
98→122: |122−98| = 24
122→124: |124−122| = 2
124→183: |183−124| = 59
183→37: |37−183| = 146
37→14: |14−37| = 23

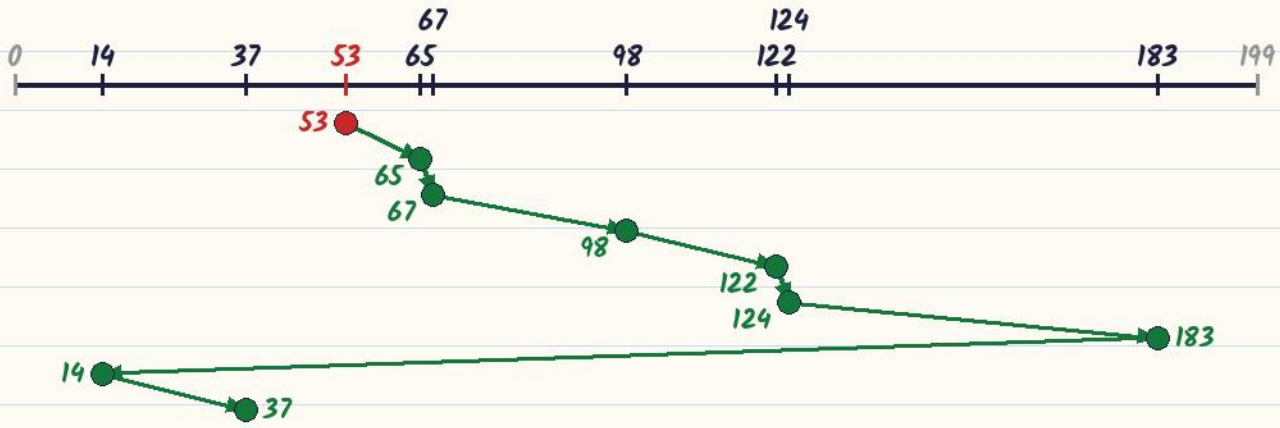
Total = 12 + 2 + 31 + 24 + 2 + 59 + 146 + 23 = 299

⚠ Trap: LOOK = SCAN − (बेकार का end तक जाना)। इसलिए LOOK ≤ SCAN। C-LOOK = C-SCAN का LOOK version (183 से सीधे 14 पर)।

3d. तुलना + C-LOOK + Practice

Bonus: C-LOOK (Circular LOOK)

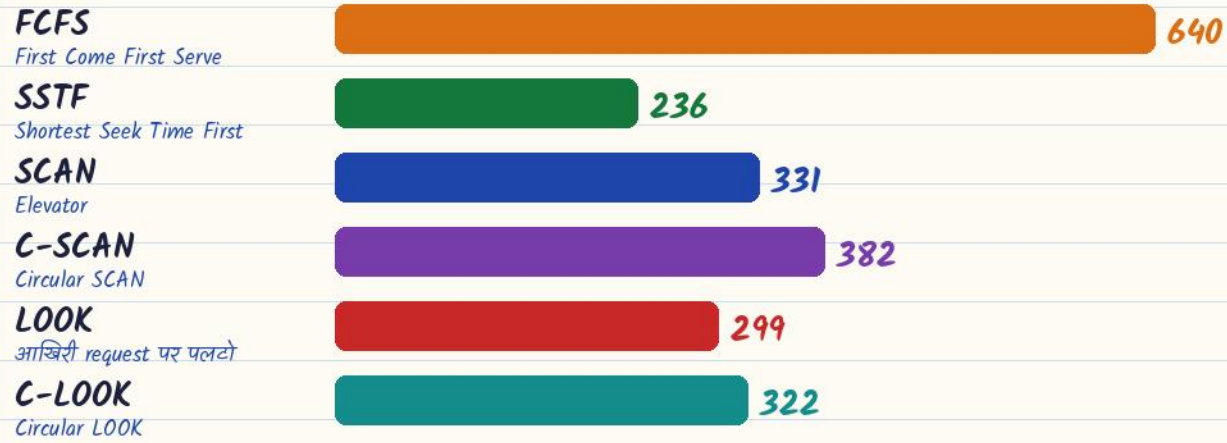
ऊपर 183 तक, फिर सीधे सबसे छोटी request 14 पर कूदो, फिर 37। (0 और 199 पर नहीं जाते)



53→65: |65−53| = 12
65→67: |67−65| = 2
67→98: |98−67| = 31
98→122: |122−98| = 24
122→124: |124−122| = 2
124→183: |183−124| = 59
183→14: |14−183| = 169
14→37: |37−14| = 23

Total = 12 + 2 + 31 + 24 + 2 + 59 + 169 + 23 = 322

Head = 53, Queue same — Total Seek Distance तुलना:



सीख:

इस example में SSTF सबसे कम (236), FCFS सबसे ज़्यादा (640)। LOOK हमेशा SCAN से ≤, C-LOOK हमेशा C-SCAN से ≤।

✍ Practice (अगले page पर पूरा हल)

Tracks 0–199, Head = 50, Queue = 82, 170, 43, 140, 24, 16, 190

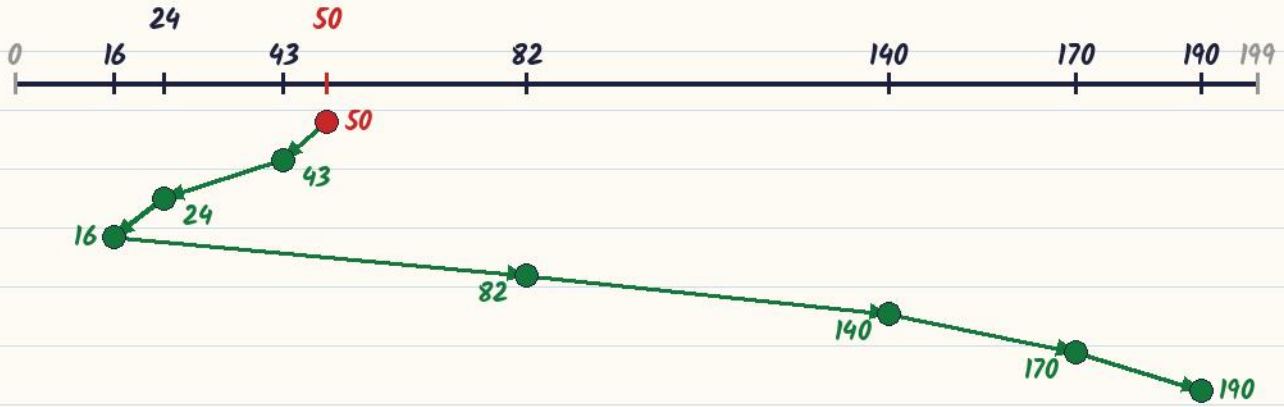
Q1. SSTF (Shortest Seek Time First) से Total Seek Distance निकालो।

Q2. SCAN से Total Seek Distance — इस बार head छोटे track (0) की तरफ़ जा रहा है।

3e. Practice Solutions (Track Diagram)

Q1. SSTF (Shortest Seek Time First) — head 50

50 से: 43 (7 दूर) सबसे पास → 24 (19) → 16 (8) → अब सिर्फ़ ऊपर बची: 82 → 140 → 170 → 190



50→43: |43−50| = 7 | 43→24: |24−43| = 19 | 24→16: |16−24| = 8

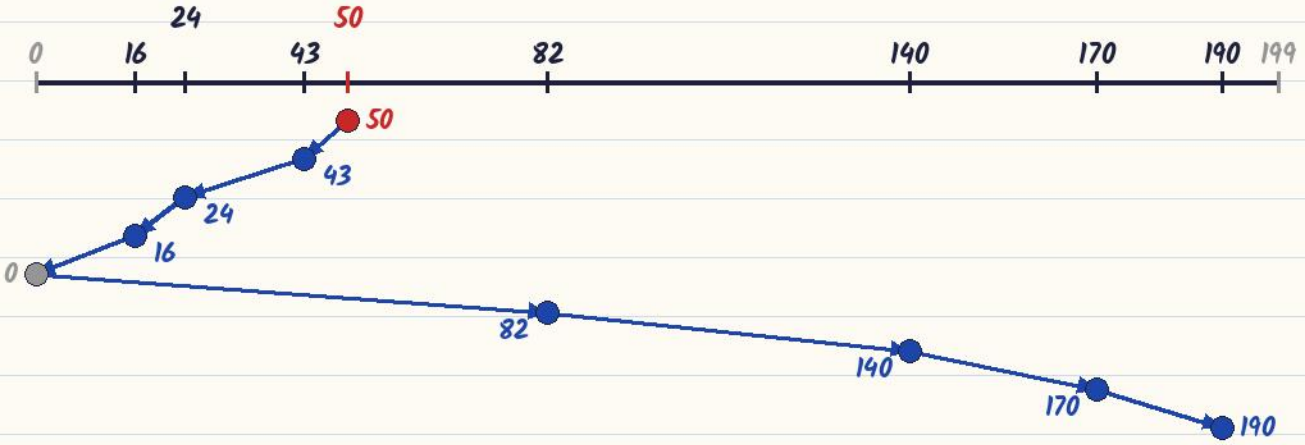
16→82: |82−16| = 66 | 82→140: |140−82| = 58 | 140→170: |170−140| = 30

170→190: |190−170| = 20

Total = 7 + 19 + 8 + 66 + 58 + 30 + 20 = 208

Q2. SCAN — head 50, नीचे (0) की तरफ़

पहले 43, 24, 16, फिर disk के end 0 तक, फिर पलटकर 82, 140, 170, 190 (190 पर रुक गए, आगे request नहीं)।



50→43: |43−50| = 7 | 43→24: |24−43| = 19 | 24→16: |16−24| = 8

16→0: |0−16| = 16 | 0→82: |82−0| = 82 | 82→140: |140−82| = 58

140→170: |170−140| = 30 | 170→190: |190−170| = 20

Total = 7 + 19 + 8 + 16 + 82 + 58 + 30 + 20 = 240

⚠ Trap: Q2 में दिशा 'नीचे' है, इसलिए end = 0 (199 नहीं)। Shortcut: (50 − 0) + (190 − 0) = 240 ✓

4. Number System: Ladder से Conversion

हर number system का एक Base होता है: Binary = 2 (0,1), Octal = 8 (0–7), Decimal = 10, Hexadecimal (Hex) = 16 (0–9, A=10 … F=15)। हर digit की कीमत = digit × base^position। Point (.) के बाएँ position 0,1,2… और दाएँ −1,−2…

(A) Binary → Decimal: हर bit के नीचे उसकी कीमत लिखो

1	1	0	1	.	1	1
2^3 =8	2^2 =4	2^1 =2	2^0 =1		2^{-1} =½	2^{-2} =¼

बाएँ: $2^0, 2^1, 2^2$…
दाएँ: $2^{-1}, 2^{-2}$…

$(1101.11)_2 = 1\times8 + 1\times4 + 0\times2 + 1\times1 + 1\times½ + 1\times¼ = 8 + 4 + 0 + 1 + 0.5 + 0.25 = (13.75)_{10}$

(B) Decimal → Binary: $(45.625)_{10}$ — integer और fraction अलग-अलग

Integer 45: बार-बार 2 से भाग, शेष लिखो

शेष (Remainder)

2	45	1
2	22	0
2	11	1
2	5	1
2	2	0
2	1	1
	0	

नीचे से ऊपर पढ़ो

Fraction .625: बार-बार 2 से गुणा

हर बार fraction × 2, पूर्णांक (integer) हिस्सा उठाओ:

0.625 × 2 = 1.25 → 1
0.25 × 2 = 0.5 → 0
0.5 × 2 = 1 → 1

ऊपर से नीचे पढ़ो

45 = 101101 और .625 = .101 ⇒ $(45.625)_{10} = (101101.101)_2$

क्यों? भाग से बचे शेष ही सबसे छोटे से बड़े bit बनाते हैं, इसलिए उल्टा (नीचे से ऊपर) पढ़ते हैं। गुणा में पहला निकला अंक ही point के ठीक बाद वाला bit है, इसलिए ऊपर से नीचे।

⚠ Trap: Integer ladder नीचे से ऊपर, Fraction ladder ऊपर से नीचे — उल्टा पढ़ा तो पूरा उत्तर गलत। Fraction कभी-कभी खत्म नहीं होता (जैसे 0.1) — तब 4–5 bit पर रुक जाओ।

रटने वाली Table (0–15) — exam में सबसे ज़्यादा काम आएगी:

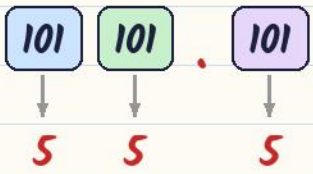
Dec	0	1	2	3	4	5	6	7
Bin	0000	0001	0010	0011	0100	0101	0110	0111
Oct	0	1	2	3	4	5	6	7
Hex	0	1	2	3	4	5	6	7

Dec	8	9	10	11	12	13	14	15
Bin	1000	1001	1010	1011	1100	1101	1110	1111
Oct	10	11	12	13	14	15	16	17
Hex	8	9	A	B	C	D	E	F

4b. Octal, Hex: Grouping Trick

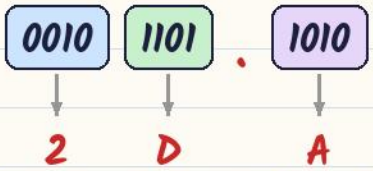
Octal का 1 digit = 3 bit (क्योंकि $2^3 = 8$), Hex का 1 digit = 4 bit ($2^4 = 16$)। इसलिए Binary ↔ Octal/Hex में भाग की ज़रूरत नहीं — बस point (.) से दोनों तरफ़ groups बनाओ।

(C) $(101101.101)_2$ → Octal: point से बाएँ और दाएँ 3-3 के group



$= (55.5)_8$

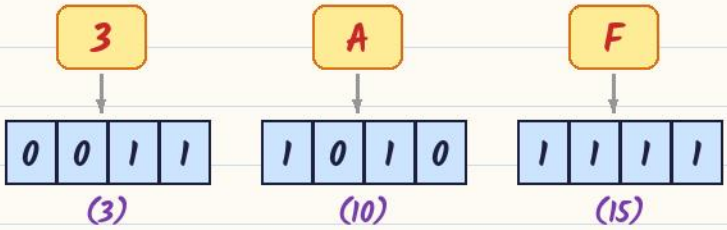
(D) Same number → Hex: 4-4 के group (कमी हो तो 0 जोड़ो)



$= (2D.A)_{16}$

बाएँ 101101 → 0010 1101 (आगे 0 जोड़े)। दाएँ .101 → .1010 (पीछे 0 जोड़ा)। 1101 = 13 = D, 1010 = 10 = A।

(E) Hex → Binary: हर digit को 4 bit में खोलो — $(3AF)_{16}$



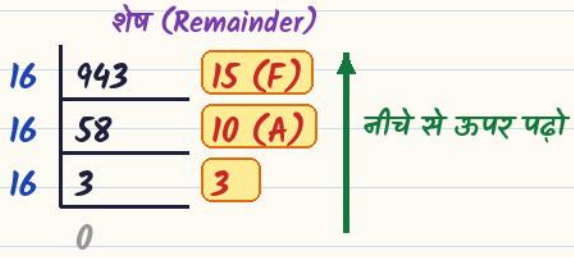
$= (0011\ 1010\ 1111)_2$

Hex → Decimal: $(3AF)_{16} = 3\times16^2 + 10\times16^1 + 15\times16^0 = 768 + 160 + 15 = (943)_{10}$

(F) Decimal → Octal / Hex: वही ladder, बस 8 या 16 से भाग



$943 = (1657)_8$

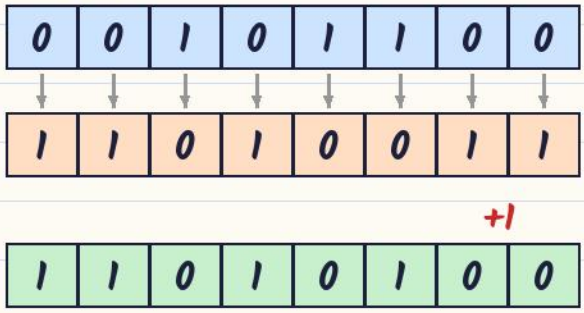


$943 = (3AF)_{16}$ ✓ (ऊपर से match)

⚠ Trap: Grouping हमेशा point से शुरू करो — बाएँ हिस्से में 0 आगे लगाओ, दाएँ हिस्से में 0 पीछे। Hex में 10–15 को A–F लिखो (शेष 15 = F, '15' नहीं)। Octal ↔ Hex सीधे नहीं: पहले Binary बनाओ।

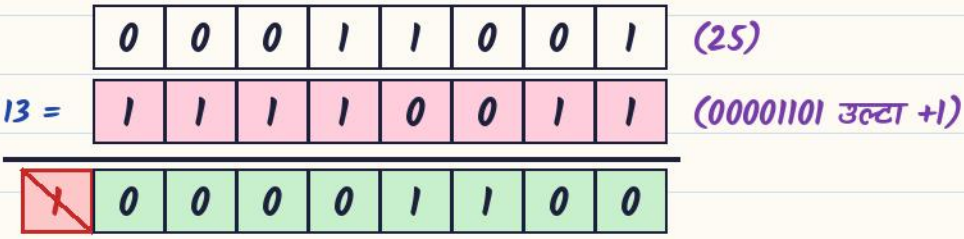
4c. 1's, 2's Complement + Binary Subtraction

Computer घटाना (subtraction) नहीं करता — वो 2's complement जोड़ देता है। 1's complement = हर bit पलट दो (0↔1)। 2's complement = 1's complement + 1। Negative number को memory में इसी रूप में रखते हैं।



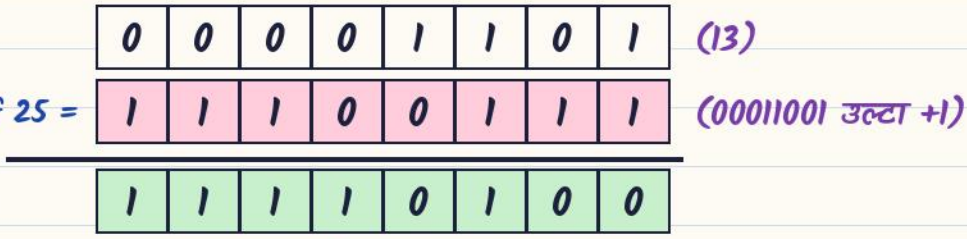
Shortcut (2's complement सीधे):
दाएँ से चलो — पहले 1 तक सब वैसा ही copy करो (उस 1 समेत), उसके बाद बाकी सब bit पलट दो। 00101100 → दाएँ से '100' वैसा, बाकी 00101 → 11010 ⇒ 11010100 ✓

(G) 25 − 13 (8-bit): A − B = A + (B का 2's complement)



Carry 1 आया → उसे फेंक दो (काटा हुआ) → result positive = 00001100 = 12 ✓

(H) 13 − 25: जब छोटे में से बड़ा घटे



Carry नहीं आया → result negative है → 11110100 का 2's complement = 00001100 = 12 → उत्तर −12 ✓

⚠ Trap: Carry आया → उसे हटाओ, उत्तर positive। Carry नहीं आया → उत्तर negative, आखिरी result का 2's complement लो और − लगाओ। n-bit में 2's complement range: -2^{n-1} से $+(2^{n-1}-1)$, जैसे 8-bit: −128 से +127।

✍ **Practice (अगले page पर पूरा हल)**
Q1. $(156.75)_{10}$ को Binary, Octal और Hex में बदलो।
Q2. 2's complement से 18 − 27 करो (8-bit)।

4d. Practice Solutions (Ladder + Groups)

Solution Q1 — Integer 156 ladder + Fraction .75 ladder



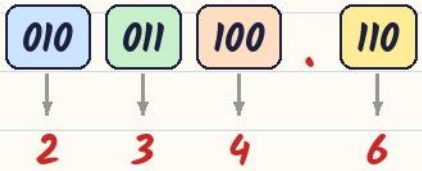
हर बार fraction × 2, पूर्णांक (integer) हिस्सा उठाओ:

$0.75 \times 2 = 1.5 \rightarrow 1$

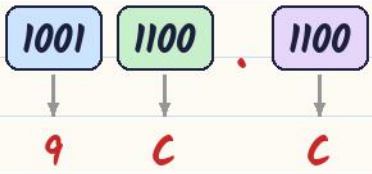
$0.5 \times 2 = 1 \rightarrow 1$

ऊपर से नीचे पढ़ो

$(156.75)_{10} = (10011100.11)_2$



3-3 group ⇒ $(234.6)_8$



4-4 group ⇒ $(9C.C)_{16}$

Check: 9×16 + 12 = 156 ✓, .C = 12/16 = .75 ✓

Solution Q2 — 18 − 27 = 18 + (27 का 2's complement)

18 =	0	0	0	1	0	0	1	0	(18)
−27 → 2's of 27 =	1	1	1	0	0	1	0	1	(00011011 उल्टा +1)
+	1	1	1	1	0	1	1	1	

Carry नहीं आया → result negative है → 11110111 का 2's complement = 00001001 = 9 → उत्तर −9 ✓

Check:
18 − 27 = −9 ✓ और −9 का 8-bit रूप = 11110111 (ऊपर वाला result) ✓

5. SOP और POS: Truth Table से Expression

SOP (Sum of Products) = AND वाले terms को OR से जोड़ो, जैसे AB' + BC। POS (Product of Sums) = OR वाले brackets को AND करो, जैसे (A+B)(B'+C)। जब हर term में सारे variable हों तो उसे Canonical (पूरा) form कहते हैं: minterm (m) और maxterm (M)।

Golden Rule (उल्टे नियम):
Minterm (m): F = 1 वाली row लो · bit 0 हो तो variable पर bar (') लगाओ · AND करो
Maxterm (M): F = 0 वाली row लो · bit 1 हो तो variable पर bar (') लगाओ · OR करो

Example: F(A,B,C) = Σm(1, 3, 5, 6, 7) — Truth table बनाओ

m	A	B	C	F	Minterm (F=1)	Maxterm (F=0)
M0	0	0	0	0	—	(A + B + C) = M0
m1	0	0	1	1	A'B'C = m1	—
M2	0	1	0	0	—	(A + B' + C) = M2
m3	0	1	1	1	A'BC = m3	—
M4	1	0	0	0	—	(A' + B + C) = M4
m5	1	0	1	1	AB'C = m5	—
m6	1	1	0	1	ABC' = m6	—
m7	1	1	1	1	ABC = m7	—

1. हरे (Minterm) column को + से जोड़ो → Canonical SOP:
   F = A'B'C + A'BC + AB'C + ABC' + ABC = Σm(1, 3, 5, 6, 7)
2. गुलाबी (Maxterm) column को गुणा (AND) करो → Canonical POS:
   F = (A + B + C)(A + B' + C)(A' + B + C) = ΠM(0, 2, 4)

क्यों उल्टा? Minterm उस एक row पर 1 देता है (A'B'C सिर्फ़ 001 पर 1)। Maxterm उस एक row पर 0 देता है ((A+B+C) सिर्फ़ 000 पर 0)। दोनों एक ही F बताते हैं।

⚠ Trap: Minterm में 0 → bar, पर Maxterm में 1 → bar। m3 = A'BC लेकिन M3 = (A + B' + C')। इन्हें उलट दिया तो पूरा उत्तर गलत।

Bonus — F' (F का complement / उल्टा):
F = Σm(1, 3, 5, 6, 7) है तो F' = Σm(0, 2, 4) (बचे हुए numbers)।
और Mi = (mi)' — यानी M0 = (A'B'C')' = A + B + C (De Morgan's law)। इसी से POS बनता है।

5b. Σm ↔ ΠM Conversion + Expand करना

3 variable में कुल $2^3 = 8$ numbers (0–7) होते हैं। जो Σm में हैं उन पर F = 1, बाकी सब पर F = 0। इसलिए ΠM = जो numbers Σm में नहीं हैं — बस 'बचे हुए' लिखो।

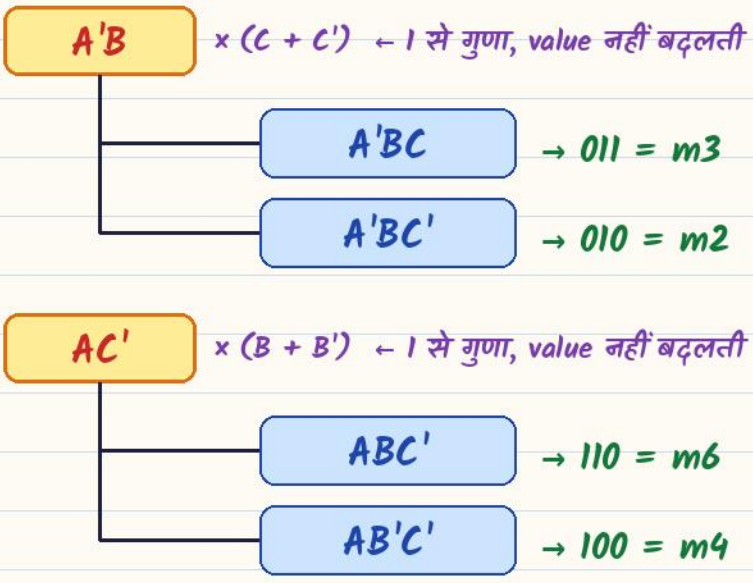
Σm(1, 3, 5, 6, 7) → 0 से 7 तक लिखो, जो नहीं हैं उन्हें ΠM में डालो:



Σm(1, 3, 5, 6, 7) = ΠM(0, 2, 4) (5 + 3 = 8 ✓ — कुल गिनती हमेशा 2^n)

Expression → Canonical: F = A'B + AC' (हर term में कोई variable गायब है)

जो variable गायब है, उसका (X + X') से गुणा करो — क्योंकि X + X' = 1, value वही रहती है, पर term 'पूरा' बन जाता है।



Result:
F = A'B + AC' = Σm(2, 3, 4, 6) = ΠM(0, 1, 5, 7)
Check: 001 डालो → A'B = 0, AC' = 0 → F = 0 ✓ (1 ΠM में है)

⚠ Trap: Σm और ΠM में same number कभी नहीं आता। Expand करने पर कोई minterm दो बार बने तो एक ही बार लिखो (A + A = A)।

✍ **Practice (अगले page पर पूरा हल)**
Q1. F(A,B,C) = Σm(0, 2, 6, 7) — Canonical POS (ΠM और पूरा expression) लिखो।
Q2. F = AB + B'C को Σm और ΠM में बदलो।

Sc. Practice Solutions (SOP / POS)

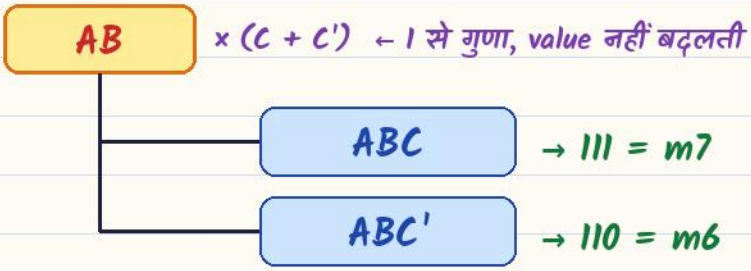
Q1. Σm(0, 2, 6, 7) → पहले 0–7 में बचे हुए निकालो

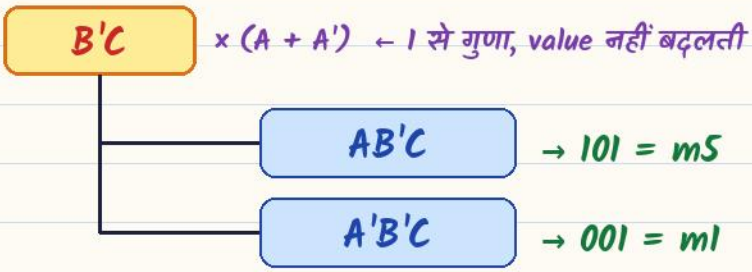


m	A	B	C	F	Minterm (F=1)	Maxterm (F=0)
m0	0	0	0	1	A'B'C' = m0	—
M1	0	0	1	0	—	(A + B + C') = M1
m2	0	1	0	1	A'BC' = m2	—
M3	0	1	1	0	—	(A + B' + C') = M3
M4	1	0	0	0	—	(A' + B + C) = M4
M5	1	0	1	0	—	(A' + B + C') = M5
m6	1	1	0	1	ABC' = m6	—
m7	1	1	1	1	ABC = m7	—

F = ΠM(1, 3, 4, 5) = (A + B + C')(A + B' + C')(A' + B + C)(A' + B + C')

Q2. F = AB + B'C → हर term expand करो





Result:

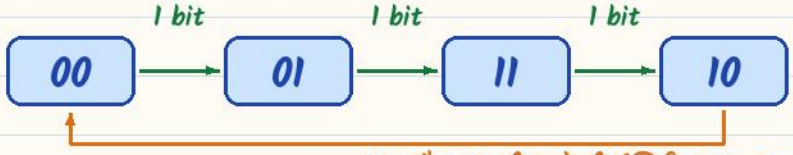
F = AB + B'C = Σm(1, 5, 6, 7) = ΠM(0, 2, 3, 4)

B'C से A'B'C (m1) और AB'C (m5) — दोनों अलग हैं, दोनों लिखो।

6. K-map (Karnaugh Map): Groups से Simplify

K-map = truth table को ऐसे खानों में सजाना कि बगल वाले खानों में सिर्फ़ 1 bit बदले (Gray code)। फिर पास-पास के 1s को 1, 2, 4, 8 के group (loop) में घेरो — जो variable group के अंदर बदल रहा है, वो हट जाता है। Group जितना बड़ा, expression उतना छोटा।

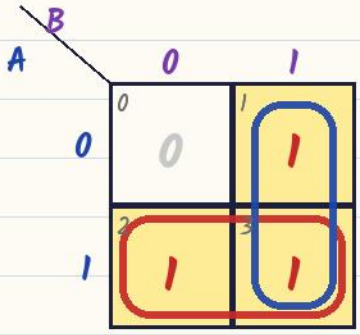
Gray code क्रम (00, 01, 10, 11 नहीं!):



Grouping के नियम:

1) Group में 1, 2, 4, 8, 16 खाने ही (3, 5, 6 नहीं)
2) सिर्फ़ सीधी लाइन या आयत (rectangle) — तिरछा (diagonal) नहीं
3) जितना बड़ा हो सके उतना बड़ा · 4) कम से कम groups
5) किनारे मुड़ते हैं: ऊपर-नीचे और बाएँ-दाएँ जुड़े, चारों कोने भी

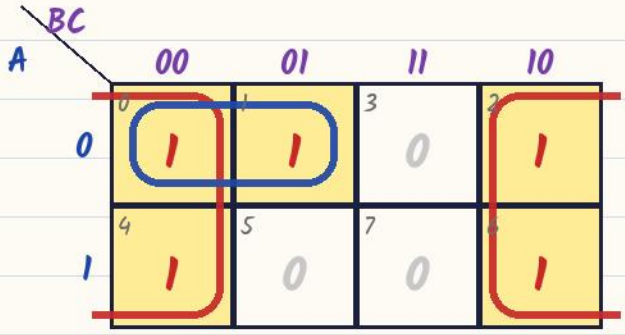
2-variable: F(A,B) = Σm(1, 2, 3)



Pair (2, 3): A=1 same, B बदला → A
Pair (1, 3): B=1 same, A बदला → B

F = A + B

3-variable: F(A,B,C) = Σm(0, 1, 2, 4, 6)



Quad (0, 2, 4, 6): C=0 same, A, B बदला → C'
Pair (0, 1): A=0, B=0 same, C बदला → A'B'

F = C' + A'B'

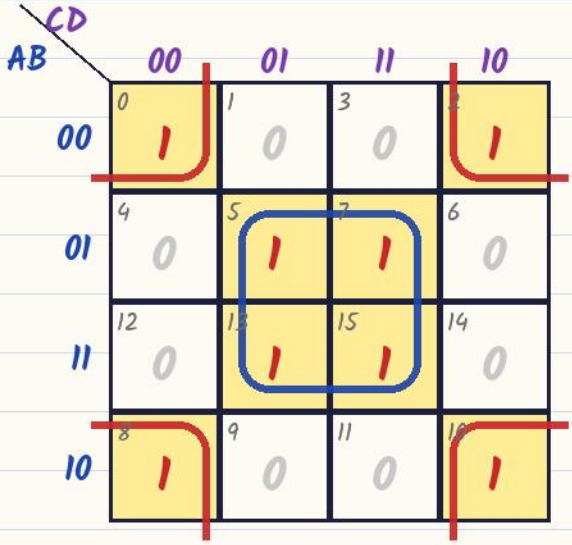
3-var में लाल loop दोनों किनारों से खुला है: column 00 और 10 (m0, m4 और m2, m6) पड़ोसी हैं — wrap-around quad। चारों में C = 0 same → C'।

⚠ Trap: 1 को दो groups में डालना allowed है (m3 और m0 ऊपर) — overlap से डरो मत। पर ऐसा group मत बनाओ जिसके सारे 1 पहले ही दूसरे groups में आ चुके हों (redundant)।

6b. 4-variable K-map: Corners, Quads, Octet

4 variable = 16 खाने। Rows = AB, Columns = CD — दोनों Gray क्रम में। हर खाने के कोने में छोटा grey अंक = minterm number (इसे रट लो: 0 1 3 2 / 4 5 7 6 / 12 13 15 14 / 8 9 11 10)।

Example 1: F = Σm(0, 2, 5, 7, 8, 10, 13, 15)



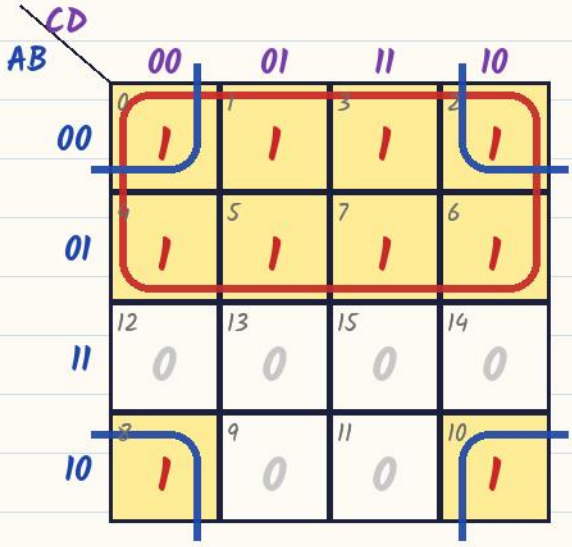
Step 1: चारों कोने (0, 2, 8, 10) = एक quad!
ऊपर-नीचे और बाएँ-दाएँ दोनों मुड़ते हैं, इसलिए corners आपस में पड़ोसी हैं।

Step 2: बीच के 5, 7, 13, 15 = दूसरा quad।

- Quad (0, 2, 8, 10): B=0, D=0 same, A, C बदला → B'D'
- Quad (5, 7, 13, 15): B=1, D=1 same, A, C बदला → BD

F = B'D' + BD

Example 2: F = Σm(0, 1, 2, 3, 4, 5, 6, 7, 8, 10)



Step 1: ऊपर की 2 पूरी rows = 8 खाने
= Octet (सिर्फ़ A = 0 same) → A'

Step 2: बचे 8, 10 → कोनों वाले quad में
(0, 2 दोबारा use) → B'D'

- Octet (0, 1, 2, 3, 4, 5, 6, 7): A=0 same, B, C, D बदला → A'
- Quad (0, 2, 8, 10): B=0, D=0 same, A, C बदला → B'D'

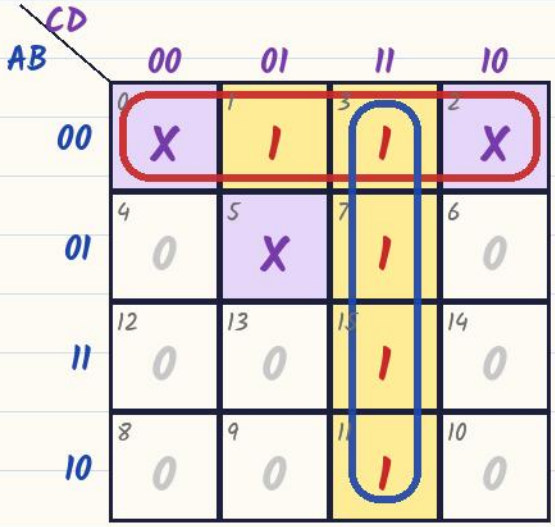
F = A' + B'D'

⚠ Trap: 8 और 10 को अलग pair (AB'D') बनाना गलत नहीं, पर बड़ा quad (B'D') लेने से 1 literal कम होता है। हमेशा पहले सबसे बड़ा group ढूँढो।

6c. Don't Care (X) और POS (0s की Grouping)

Don't care (X) = वो input जो कभी आएगा ही नहीं, इसलिए F कुछ भी हो फ़र्क नहीं पड़ता। X को 1 मान लो अगर इससे group बड़ा बनता है, नहीं तो छोड़ दो।

Example 3: F = Σm(1, 3, 7, 11, 15) + d(0, 2, 5)



X0, X2 को 1 मानो → ऊपर की पूरी row = quad → A'B'

C D = 11 वाला column = quad → CD

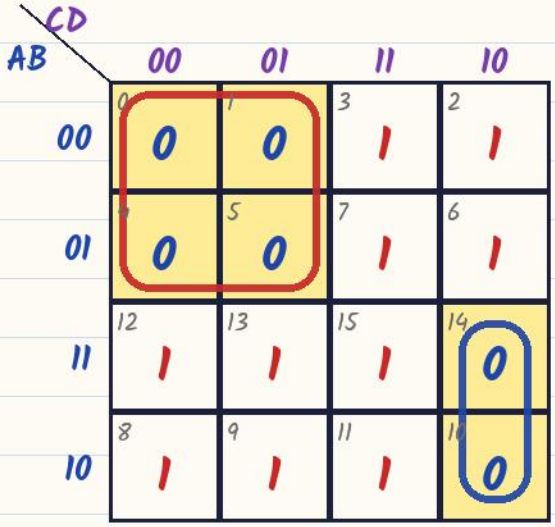
XS को छोड़ दिया (0 माना) — उससे कोई बड़ा group नहीं बनता

— Quad (0, 1, 2, 3): A=0, B=0 same, C, D बदला → A'B'

— Quad (3, 7, 11, 15): C=1, D=1 same, A, B बदला → CD

F = A'B' + CD

Example 4 POS (Product of Sums): F = ΠM(0, 1, 4, 5, 10, 14)



POS के लिए 0s के group बनाओ।

Term लिखते समय उल्टा:

0 → variable वैसा, 1 → bar (')

और term को OR (+) से लिखो, फिर सारे brackets को AND करो।

— Quad (0, 1, 4, 5): A=0, C=0 same, B, D बदला → (A+C)

— Pair (10, 14): A=1, C=1, D=0 same, B बदला → (A'+C'+D)

F = (A+C)(A'+C'+D)

⚠ Trap: POS में A = 0 same हो तो term में A (बिना bar) आता है — SOP से बिल्कुल उल्टा। Don't care को 'ज़रूरी' नहीं मानना: जो X किसी group में काम न आए, उसे घेरने की ज़रूरत नहीं।

6d. K-map Practice (खाली map में खुद भरो)

हल करने का तरीका (हर बार यही 5 step):
1) Map खींचो, Gray क्रम में labels (00 01 11 10) 2) हर minterm के खाने में 1 (या POS में 0) भरो 3) पहले Octet, फिर Quad, फिर Pair, आखिर में अकेले 1 देखो — किनारे और कोने मत भूलना 4) हर group से term: जो variable same है वही रहता है 5) सब terms जोड़ो (SOP) या गुणा करो (POS)

Q1. F(A,B,C,D) = Σm(0, 1, 2, 5, 8, 9, 10) — minimal SOP (Sum of Products)

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Hint: कोनों को देखो, और Column 00 + 01 की ऊपर-नीचे वाली rows भी।

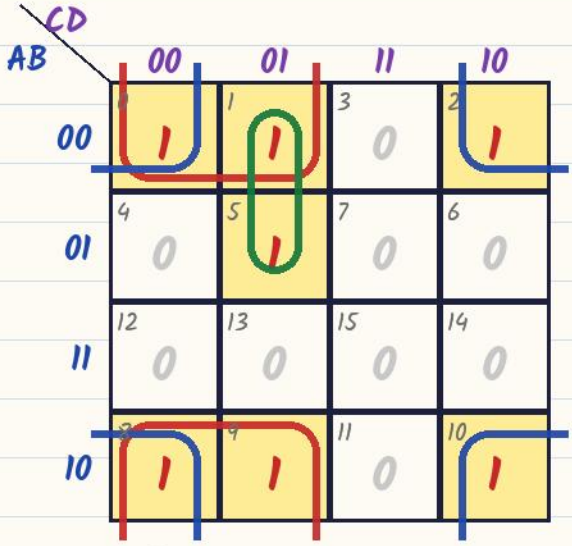
Q2. F(A,B,C,D) = ΠM(1, 3, 4, 6, 9, 11, 12, 14) — minimal POS (Product of Sums)

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Hint: यहाँ 0s भरो और 0s को घेरो। Columns 01 और 11 को ध्यान से देखो।

6e. K-map Practice Solutions

Q1. Σm(0, 1, 2, 5, 8, 9, 10)



Quad 1: rows 00 + 10 (मुड़कर), *columns 00, 01 → B'C'*

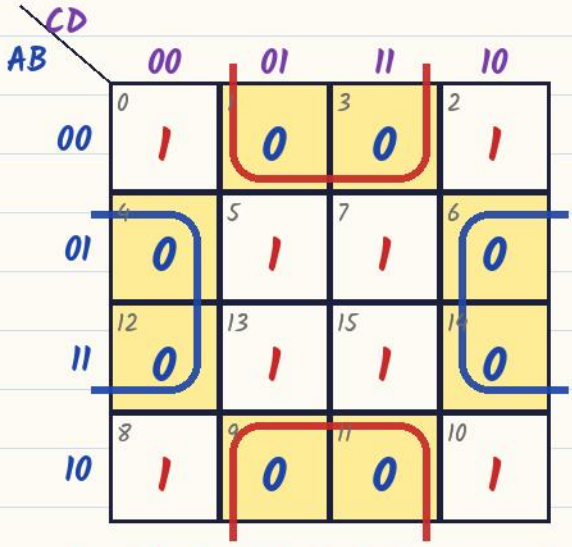
Quad 2: चारों कोने → B'D'

बचा m5: अकेला नहीं — m1 के *साथ pair (ऊपर-नीचे) → A'C'D*

- Quad (0, 1, 8, 9): B=0, C=0 same, A, D बदला → B'C'
- Quad (0, 2, 8, 10): B=0, D=0 same, A, C बदला → B'D'
- Pair (1, 5): A=0, C=0, D=1 same, B बदला → A'C'D

F = B'C' + B'D' + A'C'D

Q2. ΠM(1, 3, 4, 6, 9, 11, 12, 14) — 0s की grouping



Quad 1 (0s): 1, 3, 9, 11

B = 0, D = 1 same → (B + D')

Quad 2 (0s): 4, 6, 12, 14 —

बाएँ-दाएँ मुड़कर: B = 1, D = 0

→ (B' + D)

- Quad (1, 3, 9, 11): B=0, D=1 same, A, C बदला → (B+D')
- Quad (4, 6, 12, 14): B=1, D=0 same, A, C बदला → (B'+D)

F = (B+D')(B'+D)

Bonus: (B + D')(B' + D) = BD + B'D' = B XNOR D (दोनों same हों तो 1) !